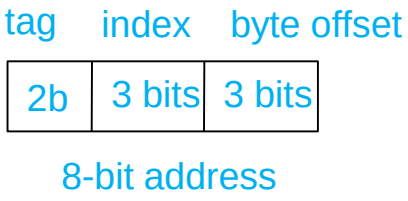
计算机体系结构

4. 高速缓存基础

李建华
计算机与信息学院

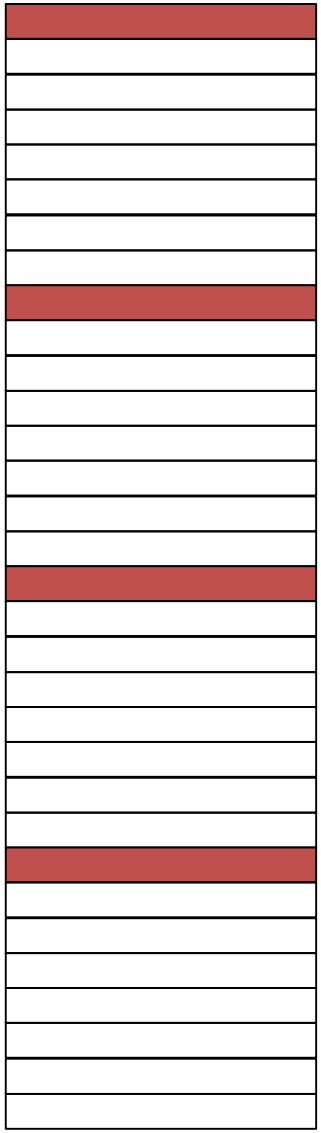
高速缓存的块和地址映射

- 内存逻辑上被划分为固定大小的块：blocks/lines
- 每个block 会被映射到缓存中的一个确定的**组(set)**，由地址中的**index bits**来确定目标组。
  - index用来定位缓存的tag和data store

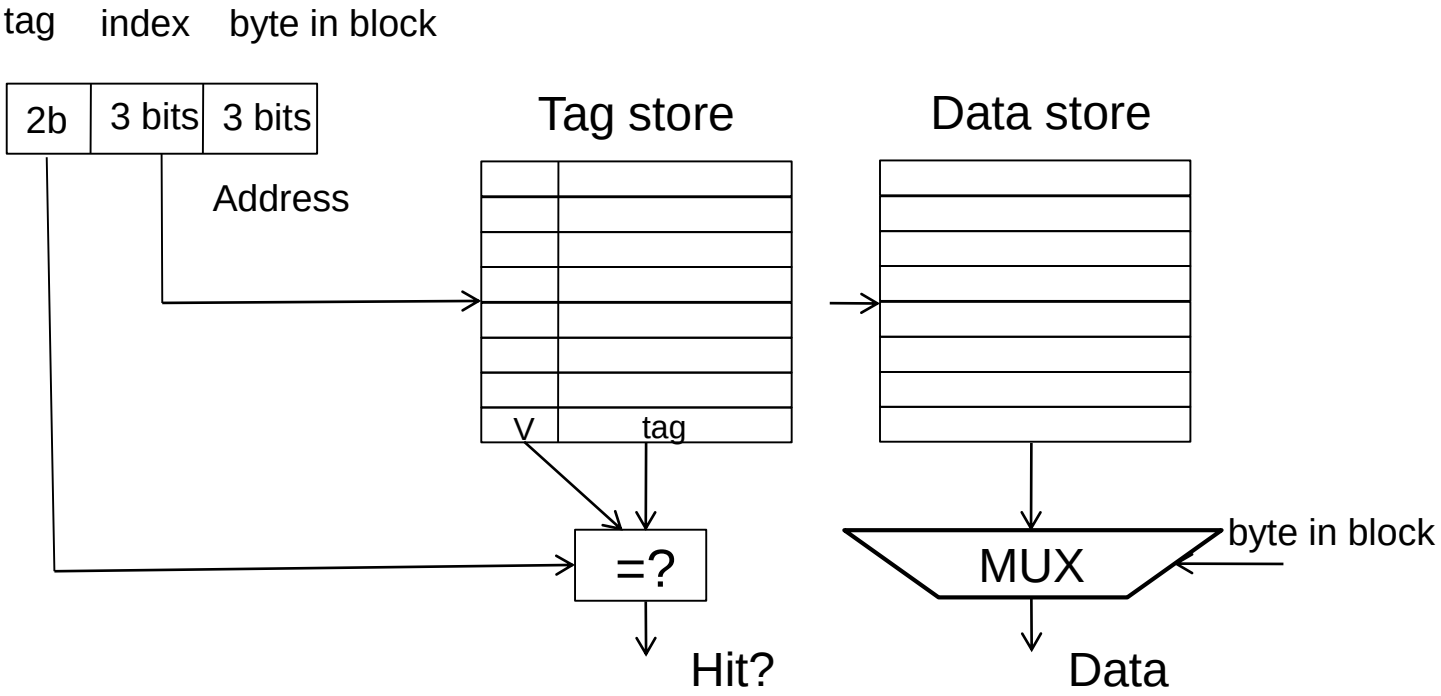


- 缓存的访问：
  - 首先，用index bits索引到可能的目标tag和data存储；
  - 随后，检查地址中的tag与tag store中对应的tag是否匹配；
  - 同时，检查tag存储中的valid bit是否有效；

直接相联缓存



- 假定一个字节寻址的存储器配置： 256 bytes, 8-byte/block → 32 blocks
- 假定高速缓存配置: 64 bytes → 8 blocks
  - 直接相联: 一个block只能保存到**唯一的**一个位置



- **问题：**具有相同index bits的block会映射到相同的缓存块，从而发生冲突。

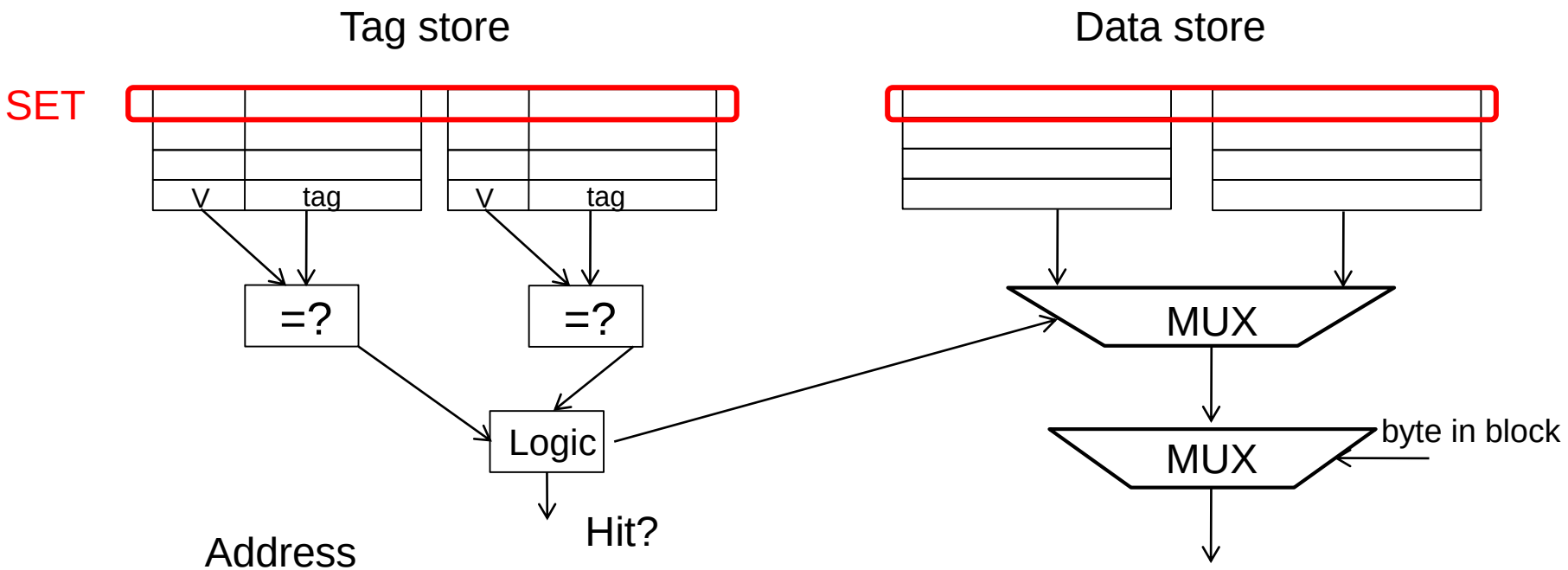
直接相联缓存

- **直接相联缓存:** 存储器中的2个block映射到缓存中的相同index所确定的同一个位置 — 不能同时容纳2个block
  - 同一个索引→ 同一个缓存位置 • • •

- 当循环访问超过1个block（均具有相同的index），就会导致缓存的命中率为0。
  - 假定地址A和B具有相同的index bits，不同的tag bits
  - 访问A, B, A, B, A, B, A, B, … → index相同会导致冲突
  - 所有的访问都是缺失（叫做：conflict misses）
  - 这个现象叫做**乒乓效应（ping-pong effect）**

2路组相联缓存

- 地址0和8在上述直接相联缓存里总是冲突
- 与其使用1列8个块的结构, 何不尝试使用2列结构?



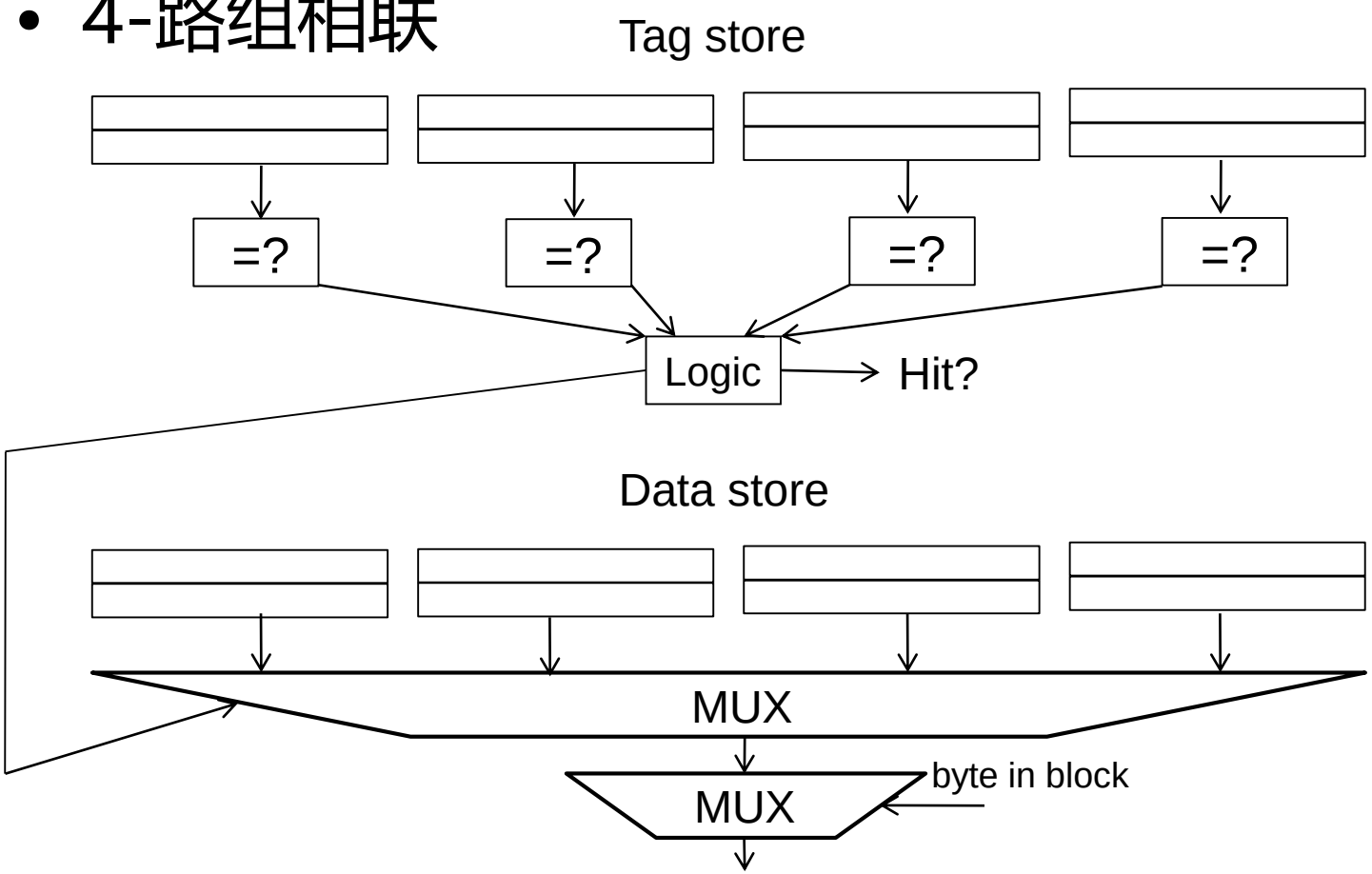
想法: 使用组的概念来设计相联存储

\+ 可以更好地容纳缓存块 -> 较少的冲突缺失

– 更加复杂，访问速度变慢，tag所需存储增加

4路组相联

- 4-路组相联

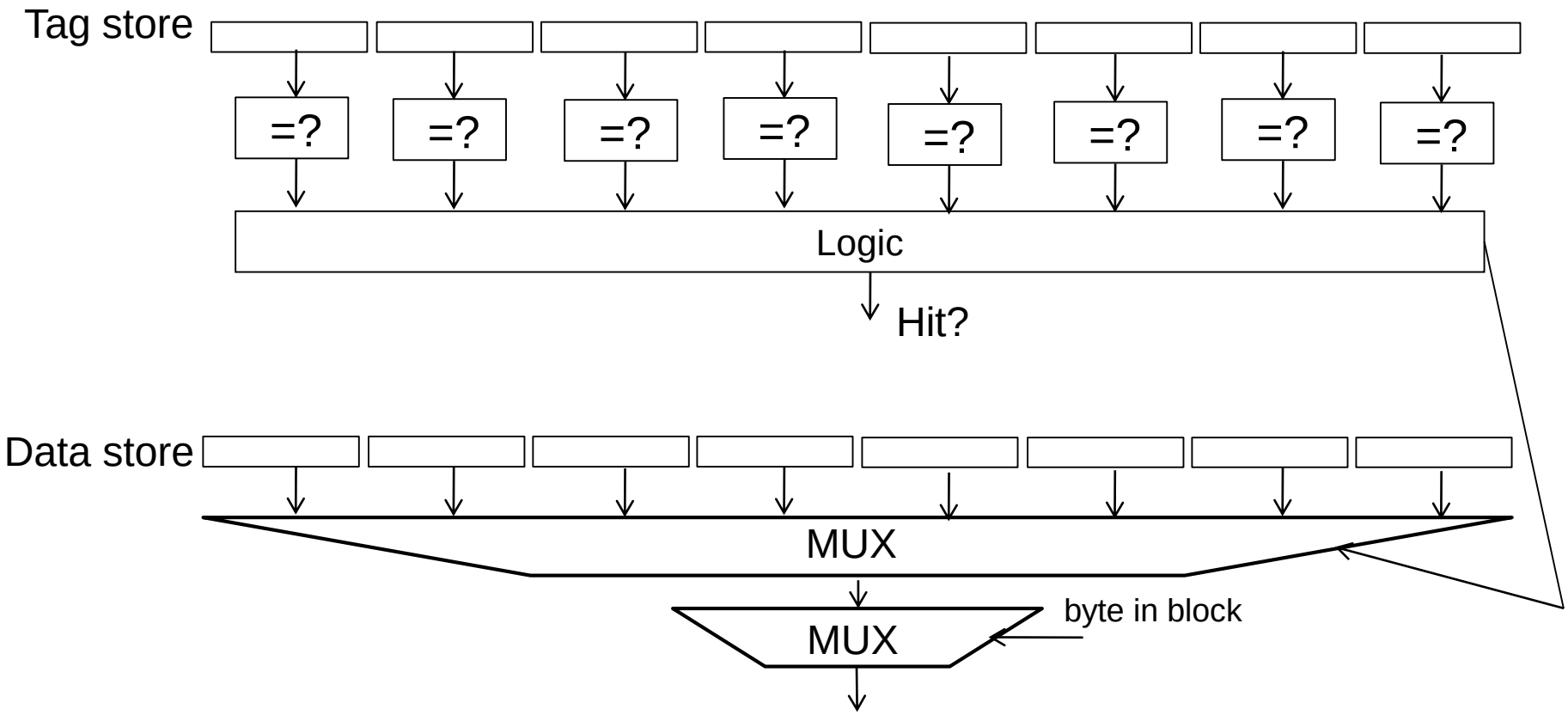


+ 发生冲突（失效）的概率进一步降低

– 更多的tag比较器，以及更宽的数据mux，更大的tag存储

全相联映射

- 全相联映射缓存
  - 每个block可以放置到缓存中的任何位置



Q：更复杂的逻辑，带来的问题是什么？

组相联缓存的主要问题

- 可以认为缓存组中每个block均具有“priority”
  - Priority是block在LRU stack中的位置
- 有三个时机，可以调整block的priority：
  - Insertion: 当访问一个缺失的block时
    - 是否将该block放入缓存?
    - 即将导入缓存的block放到什么位置?
  - Promotion: 当在cache命中一个block时
    - 是否 并 如何调整 block priority?
  - **Eviction:** 当访问缓存发生缺失且目标set已满
    - 将哪一个block进行evict？如何调整priority?

Replacement Policy

- 当发生缓存缺失并且目标set已满的时候，需要调用Replacement Policy 来替换block
  - 当然：优先使用set内的任何 invalid block。
  - 若set内所有block均valid，询问replacement策略：
    - Random
    - FIFO
    - **LRU: Least recently used (how to implement?)**
    - NRU: Not most recently used
    - LFU: Least frequently used?
    - Least costly to re-fetch?
      - Why would memory accesses have different cost?
    - Hybrid replacement policies
    - Optimal replacement policy?

https://www.gem5.org/documentation/general_docs/memory_system/replacement_policies/

Details: LRU算法的实现

- **想法:** 将组内的least recently accessed block踢出缓存。【How to?】
- **问题:** 需要维持组内block的历史访问顺序
- 对于2-路组相联缓存:
  - 如何实现LRU算法?
- 对于4-路组相联缓存:
  - 如何高效地实现LRU算法?
  - 一个组内有4个块，有多少种可能的访问顺序?
  - Q: 8-路，需要多少个bits来记录访问顺序?
  - How to choose LRU victim?

我们来看关于LRU替换策略的一个示例

Approximations of LRU

- 大多数的当代处理器没有采用高相联度缓存的“true LRU”策略
- Why?
  - True LRU 过于复杂，开销过大。
  - LRU 也只是用来预测局部性信息
    - 其未必一直或者一定是最好的策略
- 其它近似的方案:
  - Not MRU (not most recently used)
  - Hierarchical LRU
  - Victim-NextVictim Replacement
  - ……

替换算法示例：LRU or Random?

- LRU vs. Random: 哪一个性能更好?
  - 考虑: 4路组相联缓存，循环访问：A, B, C, D, E
    - LRU策略所获得的命中率为0
- Set thrashing: 当所运行的程序在一个set内的“working set” 大于相联度
  - 这时候随机策略可能更好，因为可避免抖动。
- 实际情况下，哪个替换策略更好呢?
  - 取决于workload的特征和缓存的配置
  - 其实，LRU和Random的平均命中率类似
- **更好的方案:** LRU和Random混合策略
  - 到底选择哪一个方案? Set sampling
    - See：Qureshi et al., “A Case for MLP-Aware Cache Replacement“, ISCA 2006.

有没有最优的替换策略?

Abstract:
Performance loss due to long-latency memory accesses can be reduced by servicing multiple memory accesses concurrently. The notion of generating and servicing long-latency cache misses in parallel is called Memory Level Parallelism (MLP). MLP is not uniform across cache misses - some misses occur in isolation while some occur in parallel with other misses. Isolated misses are more costly on performance than parallel misses. However, traditional cache replacement is not aware of the MLP-dependent cost differential between different misses. Cache replacement, if made MLP-aware, can improve performance by reducing the number of performance-critical isolated misses. This paper makes two key contributions. First, it proposes a framework for MLP-aware cache replacement by using a runtime technique to compute the MLP-based cost for each cache miss. It then describes a simple cache replacement mechanism that takes both MLP-based cost and recency into account. Second, it proposes a novel, low-hardware overhead mechanism called Sampling Based Adaptive Replacement (SBAR), to dynamically choose between an MLP-aware and a traditional replacement policy, depending on which one is more effective at reducing the number of memory related stalls. Evaluations with the SPEC CPU2000 benchmarks show that MLP-aware cache replacement can improve performance by as much as 23%.

Published in: 33rd International Symposium on Computer Architecture (ISCA'06)

- Qureshi et al. "A Case for MLP-Aware Cache Replacement, " in ISCA 2006.

Bonus: Cache vs Main Memory

- Main Memory (DRAM) 是磁盘/SSD的缓存
  - 通常由Virtual Memory子系统中的软件来管理
  - 需要有辅助的硬件：TLB，MMU
  - DRAM和二级存储的交换粒度：Page（页面）
- Page的替换和Block的替换非常类似
- Page Table就是物理存储所对应的“tag store”
- 二者之间的区别是什么?
  - 访问速度: cache vs. physical memory
  - Block的数量: cache vs. physical memory
  - 寻找替换对象（victim）时可以容忍的时间
  - 管理方式: hardware versus software

Tag Store中都保存了什么?

- Valid bit – 该block保存的数据是否有效
- Tag – 该block保存的数据的ID（Key）
- Replacement policy bits – priority信息
- Dirty bit – 表明该block是否被修改过
  - Used in Write-Back Cache
- Anything else?
  - 例如，多核的一致性协议状态。

写操作的处理 (1)

- 何时将修改了的数据写到下一级缓存?
  - Write-through: 当前缓存写发生的同时，写到下一级缓存;
  - Write-back: 当block从当前缓存被替换的时候写到下一级缓存。
- Write-back
  + 在被替换之前，可以合并对同一个block的多次写，从而减少缓存层级之间的带宽消耗，同时可节省能耗。
  - 标签存储中需要一个额外的dirty bit来标明 block是否被更新
- Write-through
  + 其实现简单，逻辑和硬件都较简单。
  + 所有Cache层级中的数据都是最新的，简化一致性协议的实现。
  - 与写回策略相比，需要消耗更多的带宽。

写操作的处理 (2)

- 当发生写缺失时，是否在缓存中分配一个block?
  - 写分配（write allocate）：Yes
  - 写不分配（write non-allocate）：No
- Write allocate：
  - \+ 可以聚合写操作，而不需要每次都写到下一级存储。
  - \+ 更加简单：因为写可以和读一样进行处理
  - – 需要在层级之间传输整个缓存块，可能带来问题 ???
- Write non-allocate：
  - \+ 若写操作的局部性低，可节省缓存空间 (可能具有更好的缓存命中率)
  - – 若局部性好，则性能不如write allocate

Harvard or Princeton?

- 合并架构（Princeton Architecture）的优缺点:
  - + 指令和数据可以共享缓存空间: 不需要分离设计中空间的overprovisioning （超量配置）
  - – 指令和数据可能相互竞争，没有私有空间的性能保障;
  - – 流水线中指令和数据在不同的阶段访问，合并的架构可能会导致结构冲突（structural hazard）。
- 分离架构（Harvard Architecture）：
  - – 主要是为了避免流水线的结构冲突
- 商用CPU的一级缓存绝大多数采取分离架构
- 二级和其它级别的缓存通常采用合并的方案，以提升缓存空间的利用率。

流水线与多级缓存

- 一级缓存 (instruction and data)
  - 设计的很多决定受cycle time影响
  - Small, lower associativity
  - Tag store和data store通常并行访问
- 二级缓存
  - 设计的决定需要均衡: hit rate 和 access latency
  - 通常large，highly associative; 延迟不是最重要的。
  - Tag store和data store通常顺序访问
- 层级之间的顺序和并行访问
  - Serial: 一级缓存缺失的时候，才访问二级；
  - 二级缓存所看到的访问请求和一级所看到的不同：
    - 一级缓存像是一个访问过滤器（过滤掉了一些局部性）
    - 所以，不同缓存级别所适用的管理策略不同。

高速缓存的参数与性能

- Cache size – 缓存的大小
- Block size – 缓存块的大小
- Associativity – 缓存的相联度
- Replacement policy – 缓存的替换策略
- Placement policy – 缓存的放置策略
  - 初始fill 时的Placement放置
  - 后续hit 时的Promotion调整

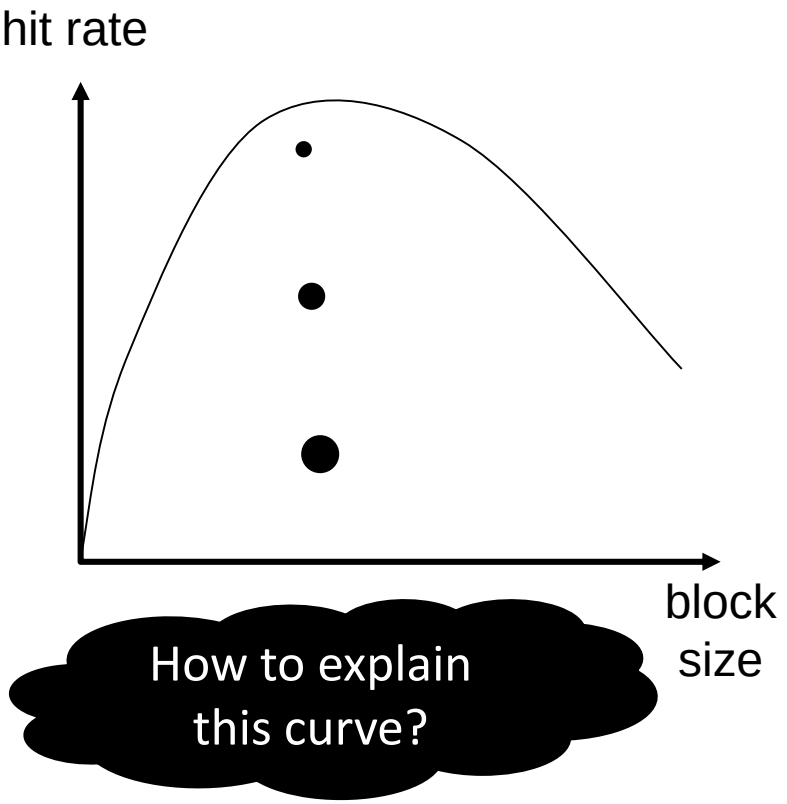
高速缓存的性能优化

Ten advanced optimizations

1. Small and simple first-level caches to reduce hit time and power
2. Way prediction to reduce hit time
3. Pipelined access and multi-banked caches to increase bandwidth
4. Non-blocking caches to increase cache bandwidth
5. Critical word first and Early restart to reduce miss penalty
6. Merging write buffer to reduce miss penalty
7. Compiler optimizations to reduce miss rate
8. Hardware prefetching to reduce miss penalty or miss rate
9. Compiler-controlled prefetching to reduce miss penalty or miss rate
10. Using HBM to extend the memory hierarchy

1. Large block size

- Block size是与地址中tag相关联的一个参数
  - 不一定非要是存储层级之间数据传输的基本粒度
    - Sub-blocking: 将一个block划分为多个pieces (每个都配置valid bit) -> 可以提升写操作性能
- 太小的block
  - 不能很好地挖掘空间局部性
  - 存储tag所需的空间开销较大
- 太大的block
  - 容量相同，Block数量过少
    - 挖掘时间局部性难度增大
  - 当空间局部性差的时候
    - 浪费空间浪费带宽，浪费功耗



Miss rate vs block size

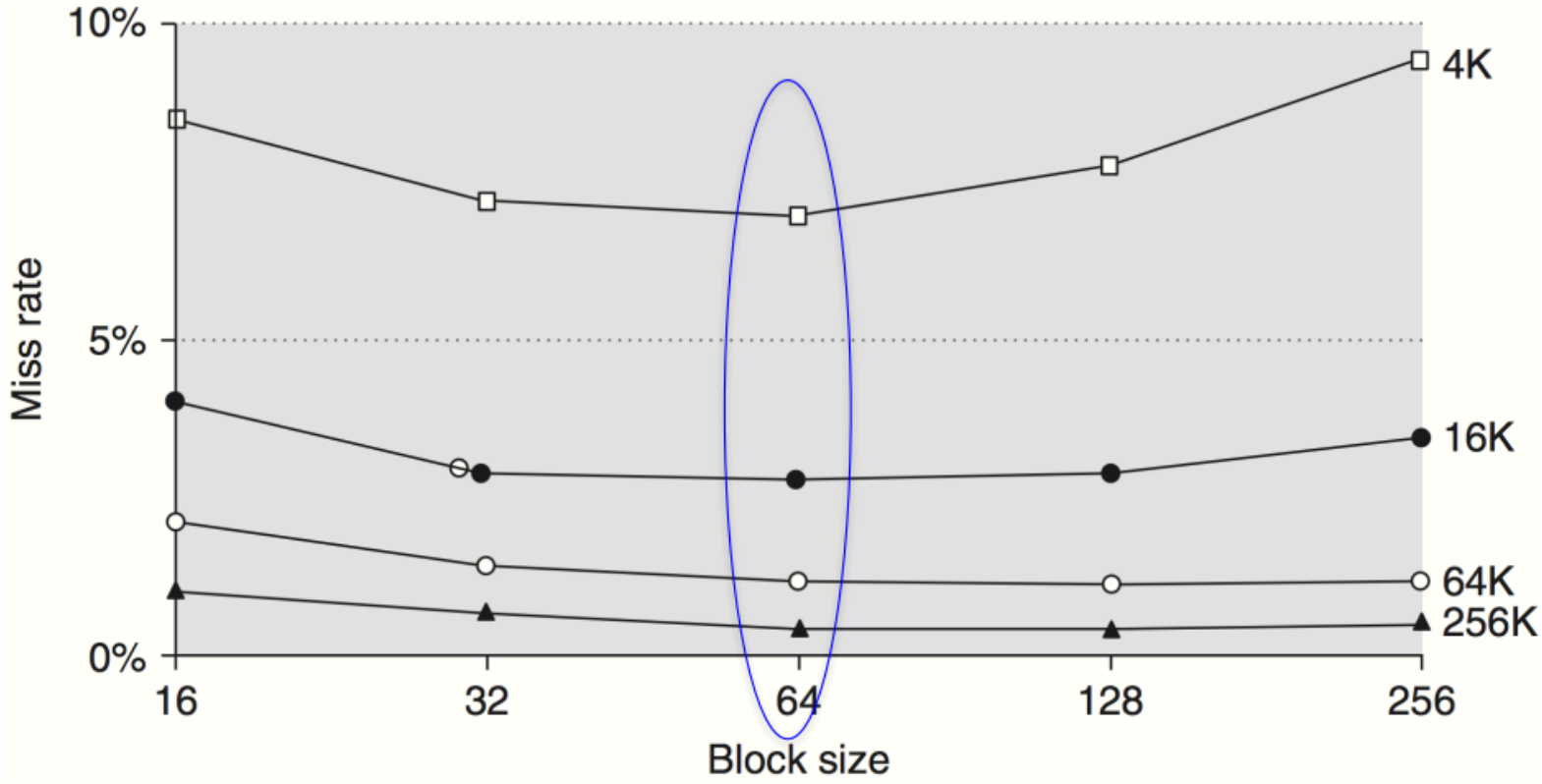


Figure B.10 Miss rate versus block size for five different-sized caches. Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. Figure B.11 shows the data used to plot these lines. Unfortunately, SPEC2000 traces would take too long if block size were included, so

Demo

	Cache size			
Block size	4K	16K	64K	256K
16	8.57%	3.94%	2.04%	1.09%
32	7.24%	2.87%	1.35%	0.70%
64	7.00%	2.64%	1.06%	0.51%
128	7.78%	2.77%	1.02%	0.49%
256	9.51%	3.29%	1.15%	0.49%

Figure B.11 Actual miss rate versus block size for the five different-sized caches in Figure B.10. Note that for a 4 KB cache, 256-byte blocks have a higher miss rate than 32-byte blocks. In this example, the cache would have to be 256 KB in order for a 256-byte block to decrease misses.

AMAT Demo

Example Figure B.11 shows the actual miss rates plotted in Figure B.10. Assume the memory system takes 80 clock cycles of overhead and then delivers 16 bytes every 2 clock cycles. Thus, it can supply 16 bytes in 82 clock cycles, 32 bytes in 84 clock cycles, and so on. Which block size has the smallest average memory access time for each cache size in Figure B.11?

Answer Average memory access time is

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

If we assume the hit time is 1 clock cycle independent of block size, then the access time for a 16-byte block in a 4 KB cache is

$$\text{Average memory access time} = 1 + (8.57\% \times 82) = 8.027 \text{ clock cycles}$$

and for a 256-byte block in a 256 KB cache the average memory access time is

$$\text{Average memory access time} = 1 + (0.49\% \times 112) = 1.549 \text{ clock cycles}$$

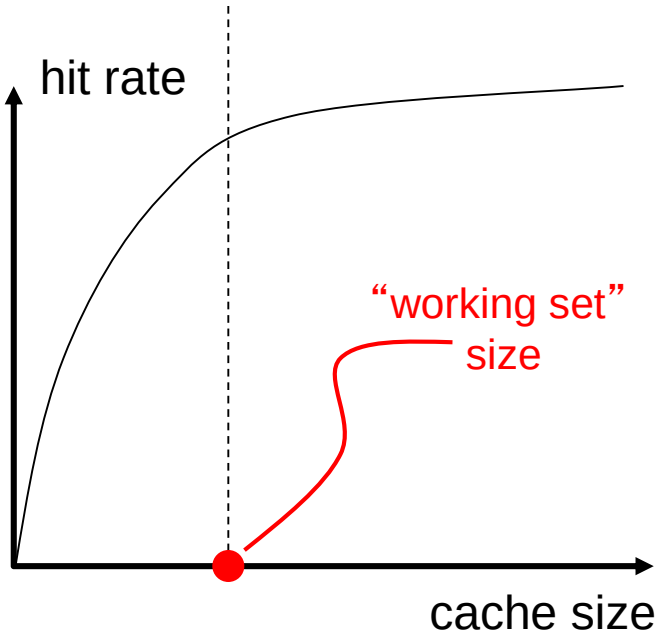
AMAT Demo

Block size	Miss penalty	Cache size 4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	**7.082**	3.411	2.134	1.588
64	88	7.160	**3.323**	**1.933**	**1.449**
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Figure B.12 Average memory access time versus block size for five different-sized caches in Figure B.10. Block sizes of 32 and 64 bytes dominate. The smallest average time per cache size is boldfaced.

2. Large cache size

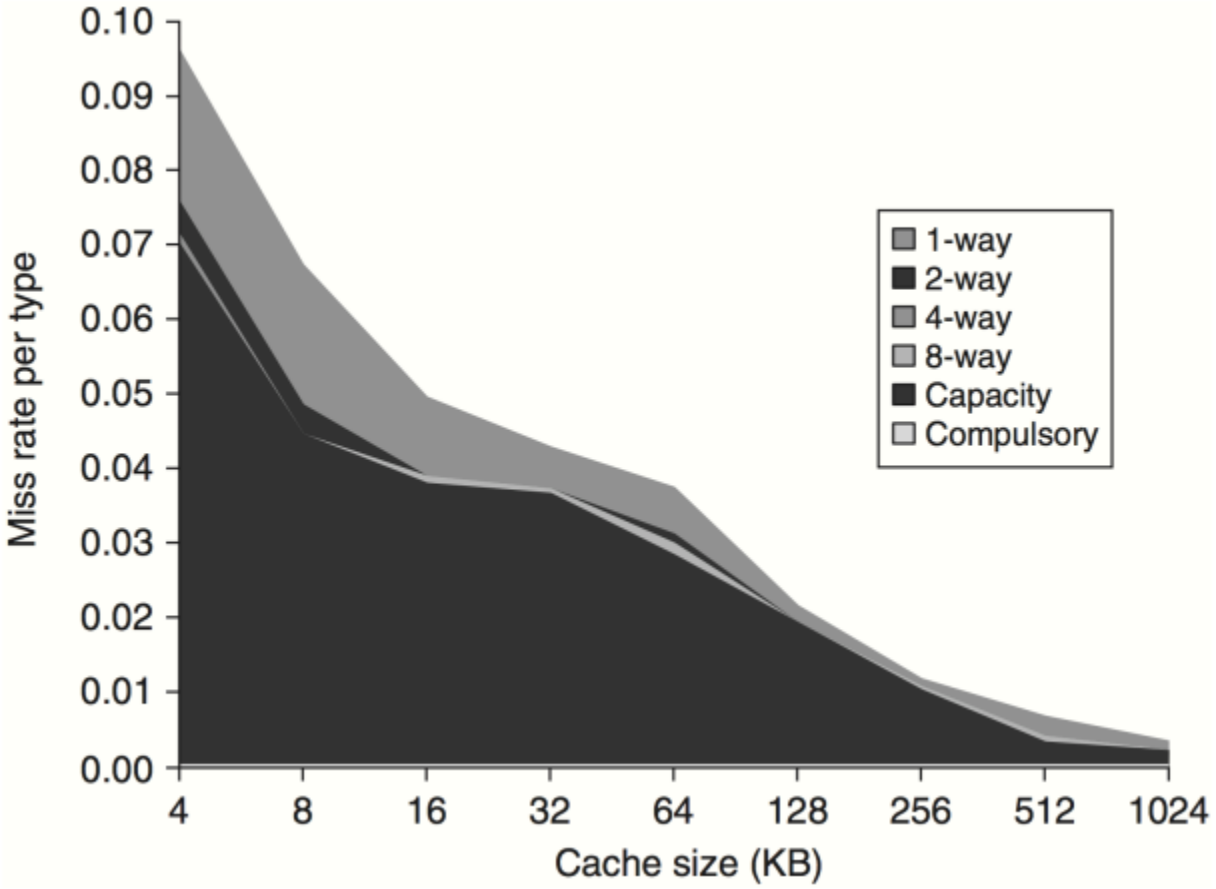
- 缓存的大小: 总的数据容量 (不包括Tag所占空间)
  - 容量越大，挖掘利用局部性的能力越强
  - 但是，容量越大 并不等价于 性能越好。
- 容量太大对访问延迟有负面影响
  - 因为：smaller is faster
  - 而访问时间可能影响关键路径
- 容量太小影响局部性的挖掘
  - 不能很好地挖掘时间局部性
  - 有用的数据可能会被频繁替换
- **Working set**: 应用程序访问的所有数据的合集
  - **注意**：是在一个时间窗口内



Peter J. Denning, "The working set model of program behavior", CACM 1968.

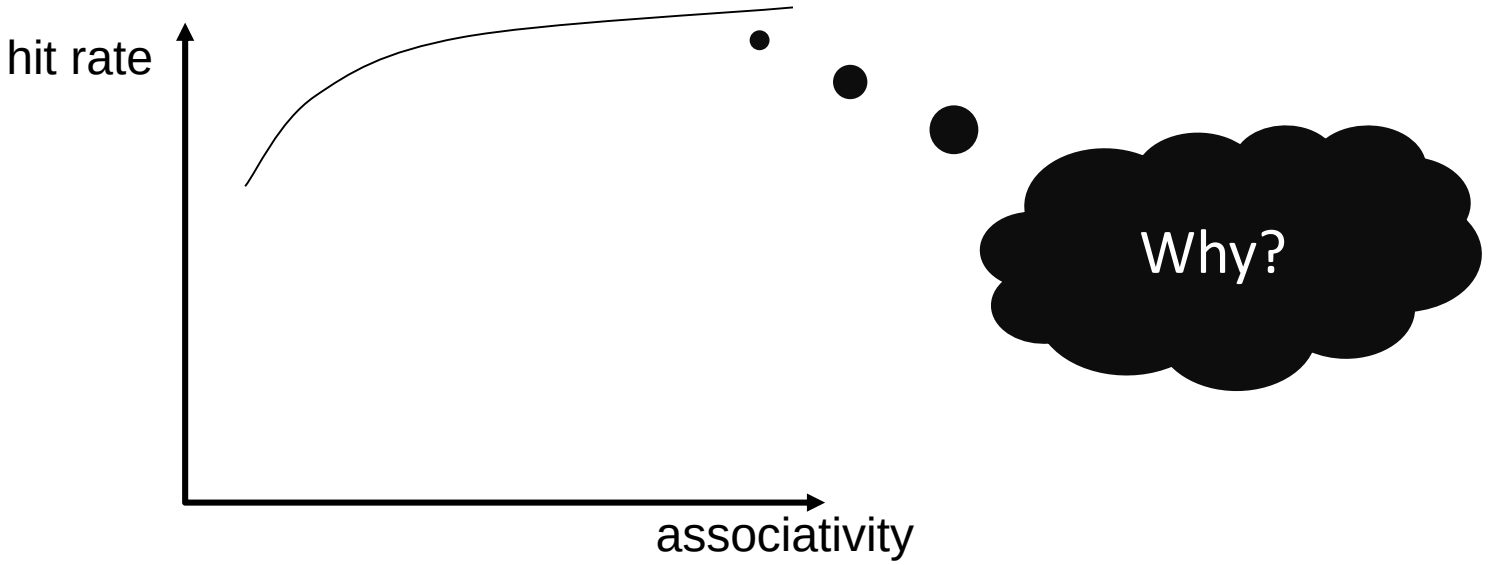
增大Cache Size的影响

- 较大的缓存，可能命中时间会增加，且开销更大；
- 目前的趋势：二级缓存和三级缓存的容量越来越大



3. High associativity

- **相联度的大小:** 多少个block可以放置到相同的组?
- 高相联度缓存的优点和缺点:
  + 较高的缓存命中率
  – **较慢的访问速度** (命中延迟 和 数据访问延迟)
  – **较大的硬件开销** (需要更多的比较器)
- 此外，高相联度带来的收益有一个衰减趋势。



More about associativity

- 低相联度缓存的特性
  - 硬件开销小
  - 访问延迟低
    - 对L1来说尤其重要
  - 缺失率高
- 相联度需要是2的整数次幂吗?
  - 不是必须的，比如有12-路的商用缓存设计

Demo

Example Assume that higher associativity would increase the clock cycle time as listed below:

$$\text{Clock cycle time}_{2\text{-way}} = 1.36 \times \text{Clock cycle time}_{1\text{-way}}$$
$$\text{Clock cycle time}_{4\text{-way}} = 1.44 \times \text{Clock cycle time}_{1\text{-way}}$$
$$\text{Clock cycle time}_{8\text{-way}} = 1.52 \times \text{Clock cycle time}_{1\text{-way}}$$

Assume that the hit time is 1 clock cycle, that the miss penalty for the direct-mapped case is 25 clock cycles to a level 2 cache (see next subsection) that never misses, and that the miss penalty need not be rounded to an integral number of clock cycles. Using Figure B.8 for miss rates, for which cache sizes are each of these three statements true?

$$\text{Average memory access time}_{8\text{-way}} < \text{Average memory access time}_{4\text{-way}}$$
$$\text{Average memory access time}_{4\text{-way}} < \text{Average memory access time}_{2\text{-way}}$$
$$\text{Average memory access time}_{2\text{-way}} < \text{Average memory access time}_{1\text{-way}}$$

	Associativity			
Cache size (KB)	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	**3.28**
8	2.69	2.58	2.55	**2.62**

Demo solution

Answer Average memory access time for each associativity is

$$\text{Average memory access time}_{8\text{-way}} = \text{Hit time}_{8\text{-way}} + \text{Miss rate}_{8\text{-way}} \times \text{Miss penalty}_{8\text{-way}}$$
$$= 1.52 + \text{Miss rate}_{8\text{-way}} \times 25$$
$$\text{Average memory access time}_{4\text{-way}} = 1.44 + \text{Miss rate}_{4\text{-way}} \times 25$$

	Associativity			
Cache size (KB)	**1-way**	**2-way**	**4-way**	**8-way**
4	3.44	3.25	3.22	**3.28**
8	2.69	2.58	2.55	**2.62**
16	2.23	**2.40**	**2.46**	**2.53**
32	2.06	**2.30**	**2.37**	**2.45**
64	1.92	**2.14**	**2.18**	**2.25**
128	1.52	**1.84**	**1.92**	**2.00**
256	1.32	**1.66**	**1.74**	**1.82**
512	1.20	**1.55**	**1.59**	**1.66**

Figure B.13 Average memory access time using miss rates in Figure B.8 for parameters in the example. Boldface type means that this time is higher than the number to the left, that is, higher associativity *increases* average memory access time.

4. Multi-level caches

- **主要思想：**

  ❶ 缓存的速度要快，以跟上 CPU 的速度；

  ❷ 缓存的容量要大，足以容纳working set

  – 所以，L1应该追求快速命中，L2追求较少的缺失

$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times \text{Miss penalty}_{L1}$$

and

$$\text{Miss penalty}_{L1} = \text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2}$$

so

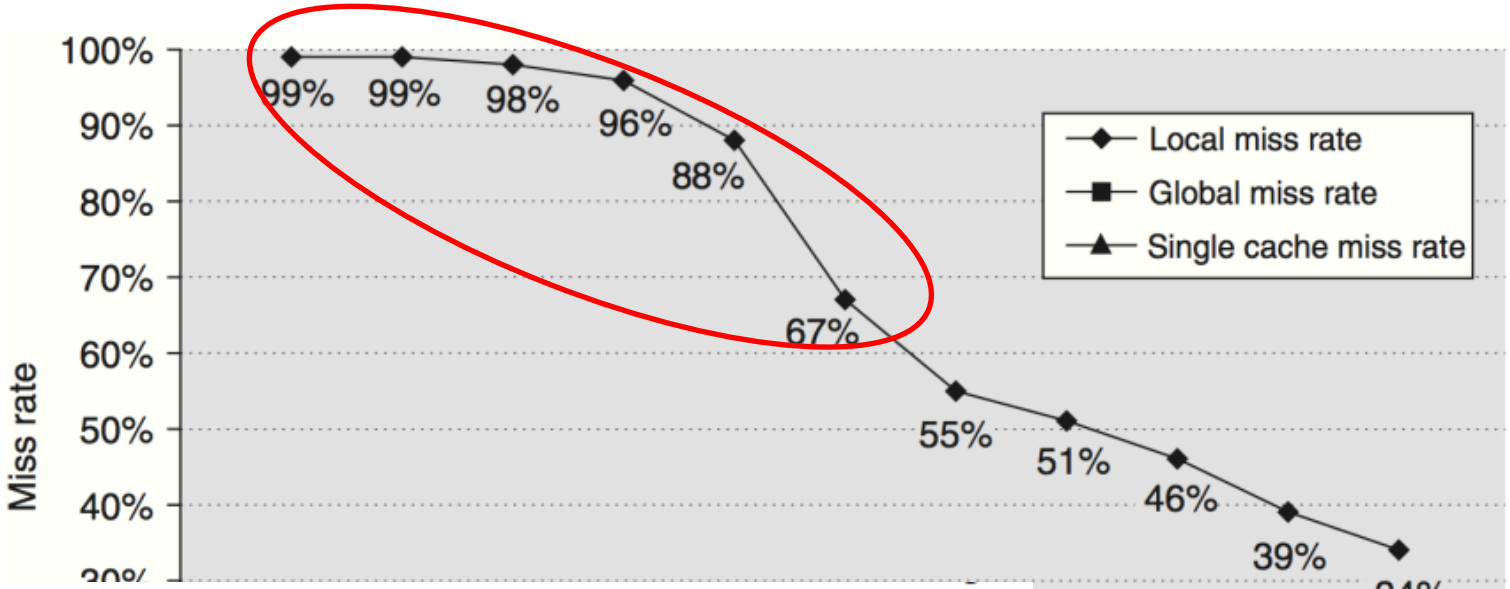
$$\text{Average memory access time} = \text{Hit time}_{L1} + \text{Miss rate}_{L1} \times (\text{Hit time}_{L2} + \text{Miss rate}_{L2} \times \text{Miss penalty}_{L2})$$

- $t_{L1} < t_{L2} < t_{L3} < t_{mem}$

局部和全局缺失率

- **局部缺失率**：该缓存中的缺失次数除以对该缓存的访问总次数（如L1$缺失率、L2$缺失率）
  - L1$会 过滤掉 存储访问中最频繁的部分
- **全局缺失率**：该缓存中的缺失次数除以 CPU 发出的访存总次数（如L1$ 缺失率、L1$ 缺失率 × L2$缺失率）
  - 表示 CPU 的内存访问中有多大比例会一直访问到该级别的存储

多级缓存中的缺失率



with the size of a second-level cache for one design. From these figures we can gain two insights. The first is that the global cache miss rate is very similar to the single cache miss rate of the second-level cache, provided that the second-level cache is much larger than the first-level cache. Hence, our intuition and knowledge about the first-level caches apply. The second insight is that the local cache miss rate is *not* a good measure of secondary caches; it is a function of the miss rate of the first-level cache, and hence can vary by changing the first-level cache. Thus, the global cache miss rate should be used when evaluating second-level caches.

Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches *smaller* than the sum of the two 64 KB first-level caches make little sense, as reflected in the high miss rates. After 256 KB the single cache is within 10% of the global miss rates. The miss rate of a single-level cache versus size is plotted against the local miss rate and

L2$ Design

- 从上一页的结论，我们可以得出L2$ Size的设计选择：
  - 由于L1$中的所有内容都可能在L2$中，所以L2$的容量应该比一级缓存大得多。
- 另外一个问题：inclusive or exclusive?
  - inclusive：一级缓存的数据始终存在于二级缓存中。
    - 优势：I/O与缓存之间的一致性维护（只需检查二级缓存）。
    - 缺点：当要替换二级缓存中的块时，二级缓存必须使所有映射到该二级块的一级缓存块失效，这会导致一级缓存缺失率略高。
    - 例如：Intel Pentium 4：一级缓存中是 64 字节的块，二级缓存中是 128 字节的块。
  - exclusive：一级缓存的数据从不会在二级缓存中找到。
    - 一级缓存缺失会导致一级缓存和二级缓存之间的块进行交换。
    - 优势：防止二级缓存空间的浪费。
    - 例如：AMD 速龙（AMD Athlon）：64KB 的一级缓存和 256KB 的二级缓存。

5. 优先处理读缺失 (1)

- **主要思想：**在写操作完成之前，处理读操作。
- 对于设置write buffer的write-through cache：

```
SW    R3, 512(R0)     ; M[512] <- R3     (cache index 0)
LW    R1, 1024(R0)    ; R1 <- M[1024]    (cache index 0)
LW    R2, 512(R0)     ; R2 <- M[512]     (cache index 0)
```

有何问题？

  - 问题：带**写缓冲**的**写直达**在**缓存缺失**时，会与主存读操作产生写后读（RAW）冲突。
    - 如果等待写缓冲为空，再到底层Memory去读，会增加读缺失处理的代价（旧的 MIPS 1000 CPU，penalty会增加 50%）
    - 更好的方法：**在读操作前检查写缓冲内容；如果没有冲突，让内存访问继续进行。**

5. 优先处理读缺失 (2)

- **想法：**在写操作完成之前，处理读操作。
- 对于write-back cache：

```
SW    R3, 512(R0)     ; M[512] <- R3     (cache index 0)
LW    R1, 1024(R0)    ; R1 <- M[1024]    (cache index 0)
LW    R2, 512(R0)     ; R2 <- M[512]     (cache index 0)
```

  - 问题：当**读缺失**会**替换一个脏块**（dirty bit set）
    - 常规做法：将脏块写入内存，然后进行读操作
      - 内存慢，引起较长的等待时间。
    - **更好的做法：将脏块复制到写缓冲，进行读操作，然后再执行写操作**
      - 因为，一旦完成读操作就会重新启动流水线，所以 CPU 停滞的时间更短。

6. 索引缓存时避免地址转换 (1)

- 缓存访问的两项任务：缓存索引 和 比较标签
- 用虚拟地址 or 物理地址 访问缓存?
  - 虚拟缓存：使用虚拟地址（VA）来访问缓存
  - 物理缓存：将虚拟地址转换成物理地址（PA）访问缓存

有何问题?

- 虚拟缓存面临的挑战：
  - **维持隔离：**必须检查页面级保护（读/写、只读、无效）
    - 它作为虚拟地址到物理地址转换的一部分进行检查
    - 解决方案：添加一个字段，从TLB复制保护信息，并在每次访问缓存时进行检查
  - **上下文切换：**不同进程的**相同虚拟地址**指向**不同的物理地址** (Homonyms)，这就要求将缓存进行flush
    - 解决方案：通过增加进程标识符标签（PID）来扩展缓存地址标签的宽度，减少不必要的flush

6. 索引缓存时避免地址转换 (2)

- 缓存访问的两项任务：缓存索引和比较标签
- 虚拟地址 or 物理地址 访问缓存
  - 虚拟缓存：使用虚拟地址（VA）来访问缓存
  - 物理缓存：在将虚拟地址转换后，使用物理地址（PA）

- 虚拟缓存面临的挑战：
  - 同义词或别名 (Synonyms)：**两个不同的VA对应同一个PA**
    - 不一致问题：虚拟缓存中存在同一数据的两个副本
    - 硬件反别名解决方案：确保每个缓存块有唯一的物理地址
      - Alpha 21264：检查所有可能的位置。如果找到，就将其失效
    - 软件页面着色解决方案：强制别名共享一些地址位
      - Sun 的 Solaris：所有别名的最后 18 位必须相同，这样就不会有重复的物理地址【变现限制可以映射的物理地址】

CCC Chair Mark D. Hill Receives The 2019 Eckert-Mauchly Award!

June 5th, 2019 / in Announcements, CCC, research horizons, Research News / by Helen Wright

The Association for Computing Machinery (ACM) and IEEE Computer Society just jointly announced that Computing Community Consortium (CCC) Chair Mark D. Hill of the University of Wisconsin-Madison is the recipient of the 2019 Eckert-Mauchly Award!

Mark D. Hill, University of Wisconsin-Madison

The Eckert-Mauchly Award was initiated in 1979 and is administered yearly by ACM and IEEE Computer Society. The award is given for contributions to computer and digital systems architecture and is considered a lifetime achievement award for computer architecture. The award was named for John Presper Eckert and John William Mauchly, who collaborated on the design and construction of the Electronic Numerical Integrator and Computer (ENIAC), the pioneering large-scale electronic computing machine, which was completed in 1947.

From the ACM press release:

> In the 1980s Hill developed the "3C" model of cache misses. A "cache miss" is an instance when data requested for processing by software or hardware is not found in the computer's cache. Cache misses can cause delays as the program or application must then access the data elsewhere. Hill's 3C model classified these misses into "compulsory misses," "capacity misses," and "conflict misses." The model was influential, as it led to important innovations such as victim caches and stream buffers, and is now a standard concept in computer architecture textbooks.
>
> Many regard Hill's work in in memory consistency models as his most significant contribution. With his student Sarita Adve, he developed SC for DRF: a consistency model using sequential consistency (SC), where data races can be avoided (data race free, or DRF). Hill's SC for DRF model has had significant impact for computer architects, especially as multiprocessors became ubiquitous and architects had to reason about which memory consistency model to use in their architectures and implementations. Years after Hill developed SC for DRF, it became the basis of Java and C++ memory models and, more recently, is being used with graphics processing units (GPUs) to understand memory consistency with heterogeneous processors.
>
> Hill's third major contribution is his work in transactional memory, a technique to minimize blocking due to critical sections. With David Wood he developed the LogTM transactional memory system, one of the first and widely-cited approaches to transactional memory. For the first time, this system enabled transactions to overrun their buffer and cache capacities, making transactions significantly easier for programmers to implement.

Next Topic

2.2 Ten Advanced Optimizations of Cache Performance

- First Optimization: Small and Simple First-Level Caches to Reduce Hit Time and Power
- Second Optimization: Way Prediction to Reduce Hit Time
- Third Optimization: Pipelined Cache Access to Increase Cache Bandwidth
- Fourth Optimization: Nonblocking Caches to Increase Cache Bandwidth
- Fifth Optimization: Multibanked Caches to Increase Cache Bandwidth
- Sixth Optimization: Critical Word First and Early Restart to Reduce Miss Penalty
- Seventh Optimization: Merging Write Buffer to Reduce Miss Penalty
- Eighth Optimization: Compiler Optimizations to Reduce Miss Rate
  - Loop Interchange
  - Blocking
- Ninth Optimization: Hardware Prefetching of Instructions and Data to Reduce Miss Penalty or Miss Rate
- Tenth Optimization: Compiler-Controlled Prefetching to Reduce Miss Penalty or Miss Rate